Week 12

K-Means Clustering & Unsupervised Discovery

What Can You Find Without Labels?

Section	Question it answers
1. From Supervised to Unsupervised	What changes when we remove the labels?
2. The K-Means Algorithm	How does k-means find clusters, and what does it assume?
3. Testing K-Means on Our Data	Can clustering recover reach directions without labels?
4. Choosing K	How do you know how many clusters exist?
5. What Clustering Cannot Find	Why does k-means fail to discover healthy vs impaired?
6. Gaussian Mixture Models	What if trials can belong to multiple clusters?
7. Bringing It Together	Where does unsupervised learning fit in the course?

1. From Supervised to Unsupervised

What changes when we remove the labels?

Every method from Weeks 5–11 was supervised: the classifier knew which trials were 0° vs 45° vs 90°, or which subjects were healthy vs impaired. But in many real clinical scenarios, labels do not exist yet. A researcher records neural population activity from stroke survivors during rehabilitation — she suspects there are subtypes of recovery, but nobody has defined what those subtypes are. Can the data itself reveal structure that no one told the algorithm to look for?

Students have already used one unsupervised method: PCA in Week 4. PCA found muscle synergy axes without any direction or impairment labels — it asked "which directions in feature space capture the most variance?" and returned continuous axes ordered by explained variance. But PCA does not find groups. It tells you "these two dimensions explain 80% of the variance" but not "there are 8 clusters in your data." Clustering is the unsupervised method that finds discrete groups.

The distinction is worth making precise. PCA asks: what are the main directions of variation? Classification (Weeks 5–11) asks: which pre-defined group does this trial belong to? Clustering asks: what groups exist in the data? All three are different questions about the same data. This week we tackle the third. Figure 1 illustrates the difference: Panel A shows the neural data with direction labels (eight clear clusters), while Panel B strips away the labels and overlays the PCA axes — PCA finds directions of variance but not the groups themselves.

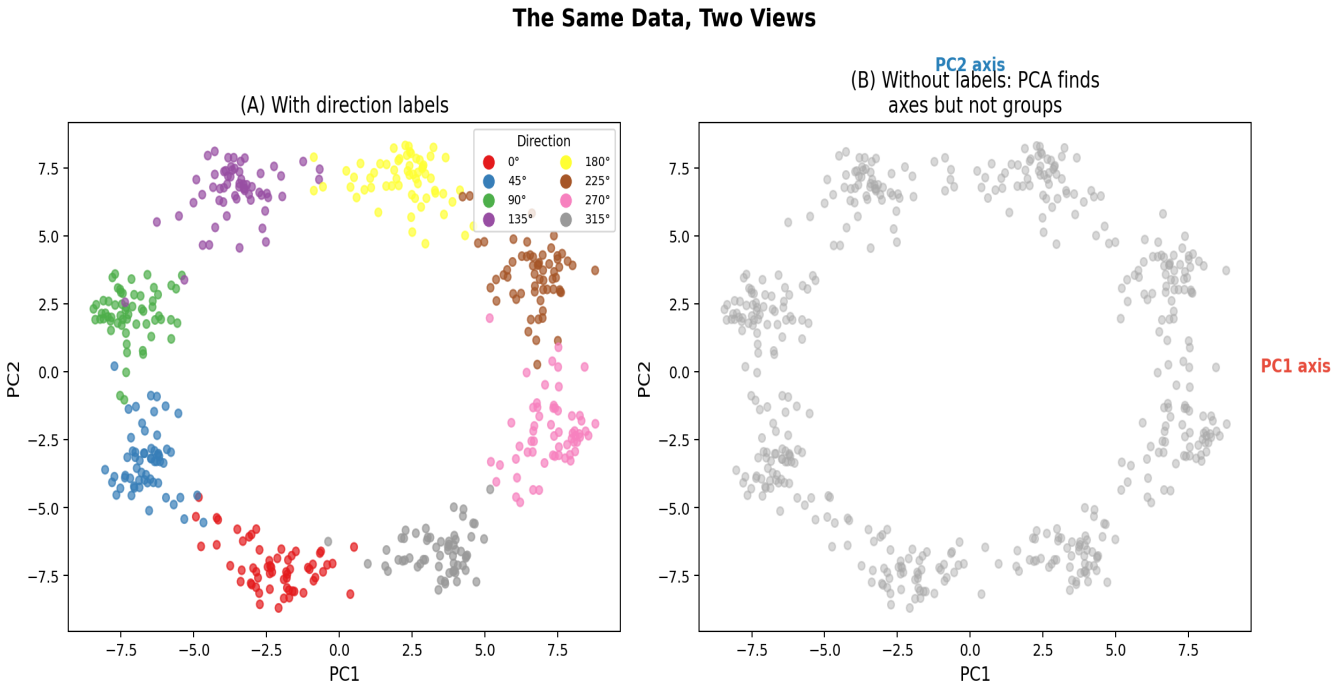

Figure 1. The same 480 neural trials in PCA space. (A) Coloured by reach direction: eight clusters are clearly visible. (B) Without labels: all trials in grey, with PCA axes overlaid. PCA finds the axes of maximum variance (as it did for EMG synergies in Week 4), but it does not identify the groups. Clustering will attempt to recover the colours from the grey.

Why do the eight clusters form a ring in PCA space? This is a direct consequence of the cosine tuning from Week 6. Each neuron's firing rate varies as a cosine function of reach direction,

peaking at its preferred direction. When 80 neurons with preferred directions spread around the circle are projected onto two principal components, the result is a circular arrangement where adjacent directions (e.g. 0° and 45°) are neighbours and opposite directions (e.g. 0° and 180°) are far apart. This structure is what makes neural data so well-suited to clustering — the directions are naturally separated in the high-dimensional space.

2. The K-Means Algorithm

How does k-means find clusters, and what does it assume?

K-means is the simplest and most widely used clustering algorithm. The user specifies K (the number of clusters to find), and the algorithm discovers where those clusters are. Here is how it works on our neural reaching data:

- Step 1 — Initialise. Place K=8 centroids in the 80-dimensional neural space. Each centroid is a candidate cluster centre. With k-means++ initialisation, the first centroid is chosen randomly, and subsequent centroids are placed far from existing ones to avoid starting with two centroids in the same cluster.
- Step 2 — Assign. For each of the 480 trials, compute the Euclidean distance to all 8 centroids. Assign the trial to its nearest centroid. This partitions the data into 8 groups — the same distance metric we used in KNN (Week 8), but here we measure distance to centroids rather than to other trials.
- Step 3 — Update. Recompute each centroid as the mean of all trials assigned to it. The centroid moves to the centre of its cluster.
- Step 4 — Repeat. Alternate between assignment (Step 2) and update (Step 3) until no trial changes its assignment. Convergence is guaranteed and typically takes 5–20 iterations.

Figure 2 shows this process on our neural data projected to 2D. Eight centroids are placed at random starting positions. Some start far from any cluster; one happens to land near the centre of the data. Despite these arbitrary starting positions, the algorithm converges in just 9 iterations — each centroid migrates to the centre of its nearest cluster, and the final assignments match the eight direction groups.

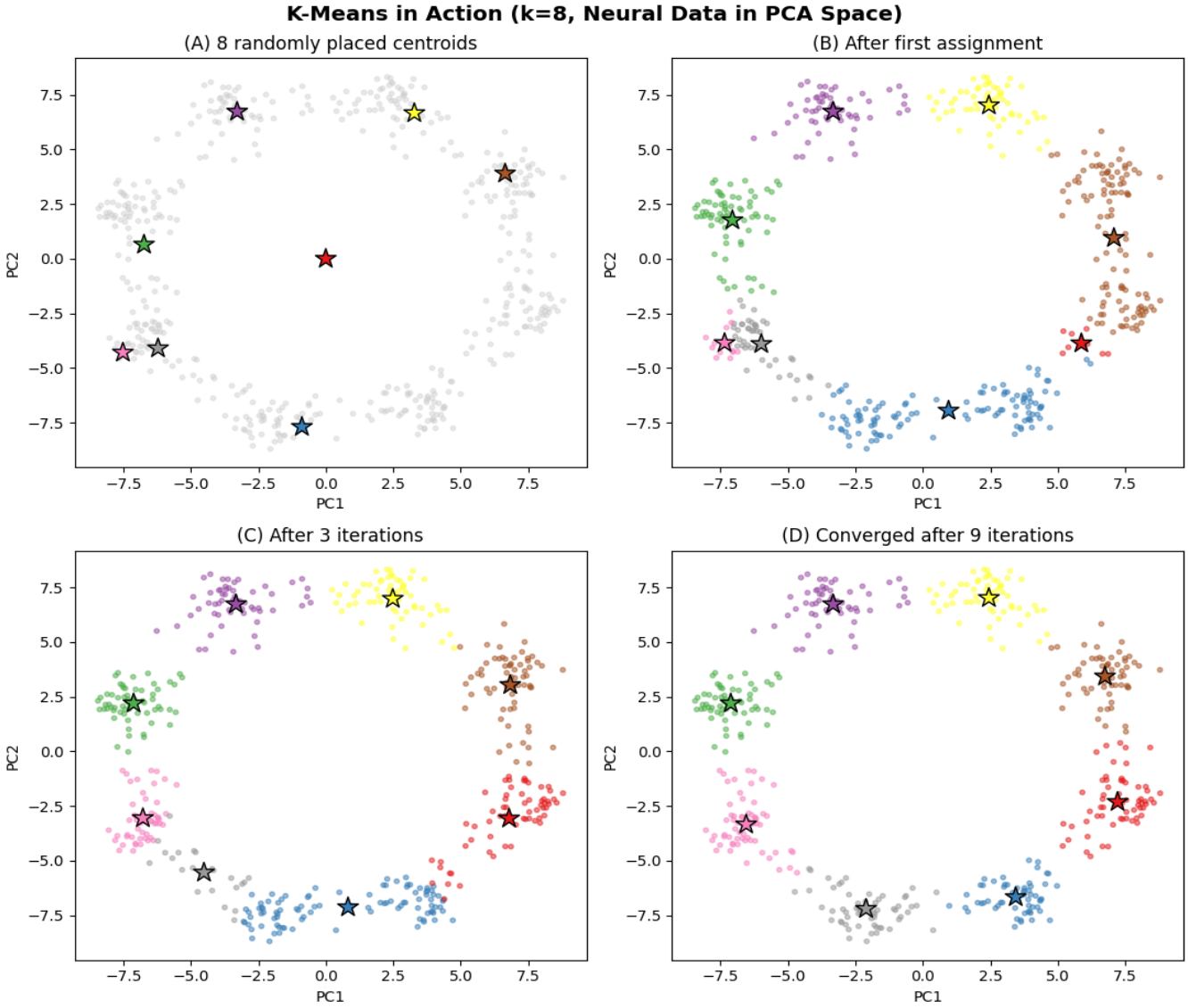

Figure 2. K-means in action (k=8, neural data in PCA space). (A) Eight randomly placed centroids (stars). (B) After the first assignment, each trial is coloured by its nearest centroid. (C) After 3 iterations, centroids have migrated toward the true cluster centres. (D) Converged after 9 iterations: centroids are stable and clusters match the true directions.

Notice that the algorithm requires the user to specify K before it begins. This is not a random guess — it should come from an understanding of the underlying process. In our case, we set K=8 because the experiment had 8 reach targets, so we expect 8 clusters of neural activity. A motor control researcher studying gait might set K=4 (walk, jog, run, sprint) based on clinical observation. A neurologist looking for impairment subtypes might try K=2 or K=3 based on a hypothesis that there are distinct recovery profiles. The point is that K encodes a scientific question: “how many distinct patterns do I believe exist in these data?” The algorithm then tests whether the data support that number. Sometimes, though, you genuinely do not know how many groups exist — that is often the whole reason for running clustering in the first place. In those cases you must either make an educated guess based on domain knowledge, or try several values of K and see which produces the most interpretable result. Section 4 introduces two heuristic methods (the elbow and silhouette) that attempt to estimate K from the data, though as we will see, they do not always find the right answer.

K-means does not explicitly assume anything about cluster shape, but its objective — minimise the total squared Euclidean distance from each trial to its centroid — works best when clusters are roughly spherical and similar in size. Euclidean distance treats all directions equally, so the algorithm naturally carves the space into round regions. And because each trial is assigned to whichever centroid is closest, a very large cluster can be split in two while two small neighbouring clusters can be merged into one. In practice, this means k-means performs well on our cosine-tuned neural data (where the 8 direction clusters are roughly equal in size and spread) but can struggle on data with elongated or unequal clusters. Section 6 introduces Gaussian Mixture Models, which explicitly model each cluster’s shape and size.

3. Testing K-Means on Our Data

Can clustering recover reach directions without labels?

We apply k-means with k=8 to the 80-dimensional neural features (480 trials from 20 subjects) and compare the cluster assignments with the true direction labels. To quantify the match, we use the Adjusted Rand Index (ARI): 1 means the clustering perfectly matches the labels (up to relabelling), and 0 means no better than random. Note that ARI requires ground-truth labels to compute — it is an evaluation tool for when you have labels and want to check whether clustering recovers them, not a tool for truly label-free analysis. The result: ARI = 0.92 — k-means nearly perfectly recovers the 8 reach directions without ever seeing the labels during training. The cosine tuning of the 80 neurons creates well-separated clusters that the algorithm finds easily.

What happens with fewer clusters? Setting k=4 forces the algorithm to merge direction pairs. Figure 3B shows which pairs merge: 0°+315°, 45°+90°, 135°+180°, and 225°+270°. These are adjacent directions with similar muscle activation patterns — exactly the groupings a biomechanist would predict. K-means does not know about muscles or biomechanics; it discovers this structure purely from the neural firing rates.

The ARI analysis is instructive here. If we compare the 4 clusters against the original 8 direction labels, ARI = 0.57 — seemingly mediocre. But that comparison is unfair: 4 clusters cannot possibly recover 8 categories. If instead we define 4 direction-pair labels (grouping each pair of adjacent directions that merged) and compare against those, ARI = 0.96 — nearly perfect. The lesson: ARI depends on what you compare against, and a “low” ARI may simply mean your reference labels are at the wrong granularity, not that the clustering failed.

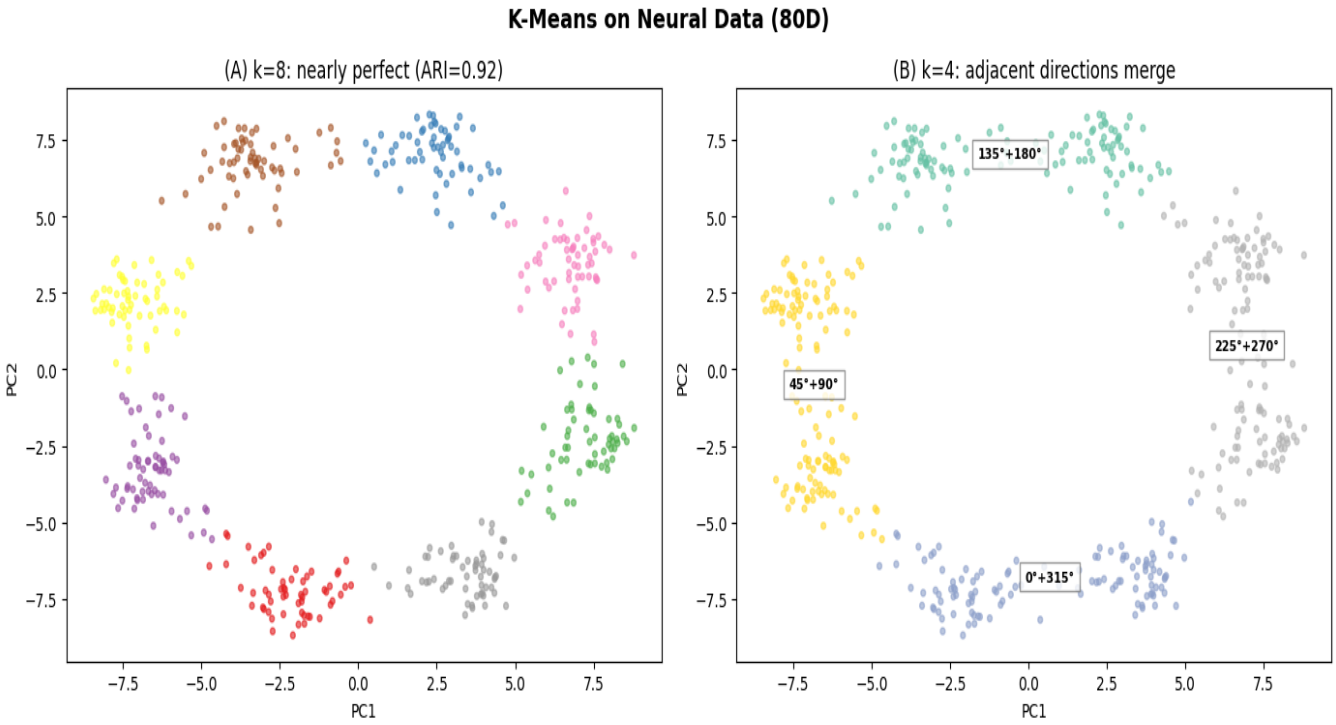

Figure 3. K-means on neural data. (A) k=8: cluster assignments in PCA space nearly match the true 8 directions (ARI=0.92). (B) k=4: the algorithm merges adjacent direction pairs into biomechanically sensible groups — annotated with the merged directions.

The story is different for EMG. Applying k-means with k=8 to the 6 muscle features gives ARI = 0.54 — roughly half the directions are confused. Why? With only 6 features, the muscle activation space is too low-dimensional to separate 8 directions cleanly. Multiple directions produce similar synergy patterns (as we saw in Week 4), so the clusters overlap. This mirrors the classification results from earlier weeks, where neural features consistently outperformed EMG for direction decoding. Figure 4 places the two results side by side.

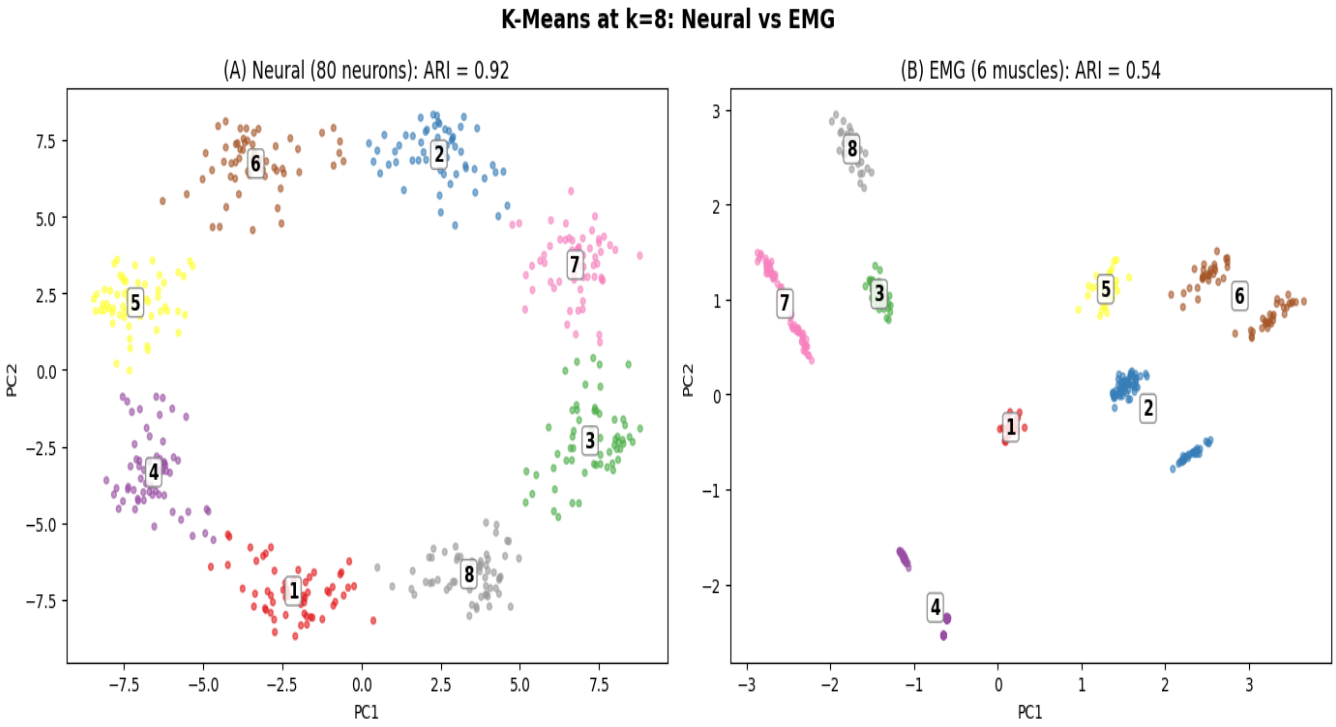

Figure 4. Neural vs EMG clustering at k=8. (A) Neural data: clean clusters (ARI=0.92). (B) EMG data: overlapping clusters (ARI=0.54). The 6-dimensional muscle space cannot separate 8 directions as cleanly as the 80-dimensional neural space.

4. Choosing K — The Hardest Problem in Clustering

How do you know how many clusters exist when you don’t have labels?

In the experiments above, we set k=8 because we knew there were 8 directions. But in a real unsupervised analysis, you do not know k in advance — that is precisely what you are trying to discover. Two popular heuristics attempt to estimate k from the data.

The elbow method plots the total within-cluster sum of squares (inertia) vs k. As k increases, inertia always decreases (more clusters = tighter fit). The idea is to look for a “bend” where adding another cluster stops helping much. Figure 5A shows the result on our neural data: inertia decreases smoothly with no clear bend. The elbow method fails here because the 8 direction clusters are roughly equally spaced, so each additional cluster captures about the same amount of variance.

The silhouette score measures, for each trial, how much closer it is to its own cluster than to the nearest other cluster. We used silhouette scores in Week 10 to compare PCA and LDA projections; now we use them to evaluate clustering quality. Figure 5B shows silhouette peaks at k=3, not k=8. Why? Silhouette rewards compact, well-separated groups. With 8 directions arranged in a circle, adjacent clusters overlap — splitting into 3 broad groups produces higher separation than 8 tightly packed groups.

The lesson is humbling: neither heuristic recovers the true k=8. Without labels, there is no objective answer to “how many clusters?” The right k depends on what granularity of structure you care about. A BCI engineer needs k=8 (individual directions). A biomechanist studying synergy groupings might prefer k=4 (direction pairs). A researcher asking “are reaches fundamentally different from returns?” might use k=2. The algorithm finds structure; a human must decide whether that structure is meaningful.

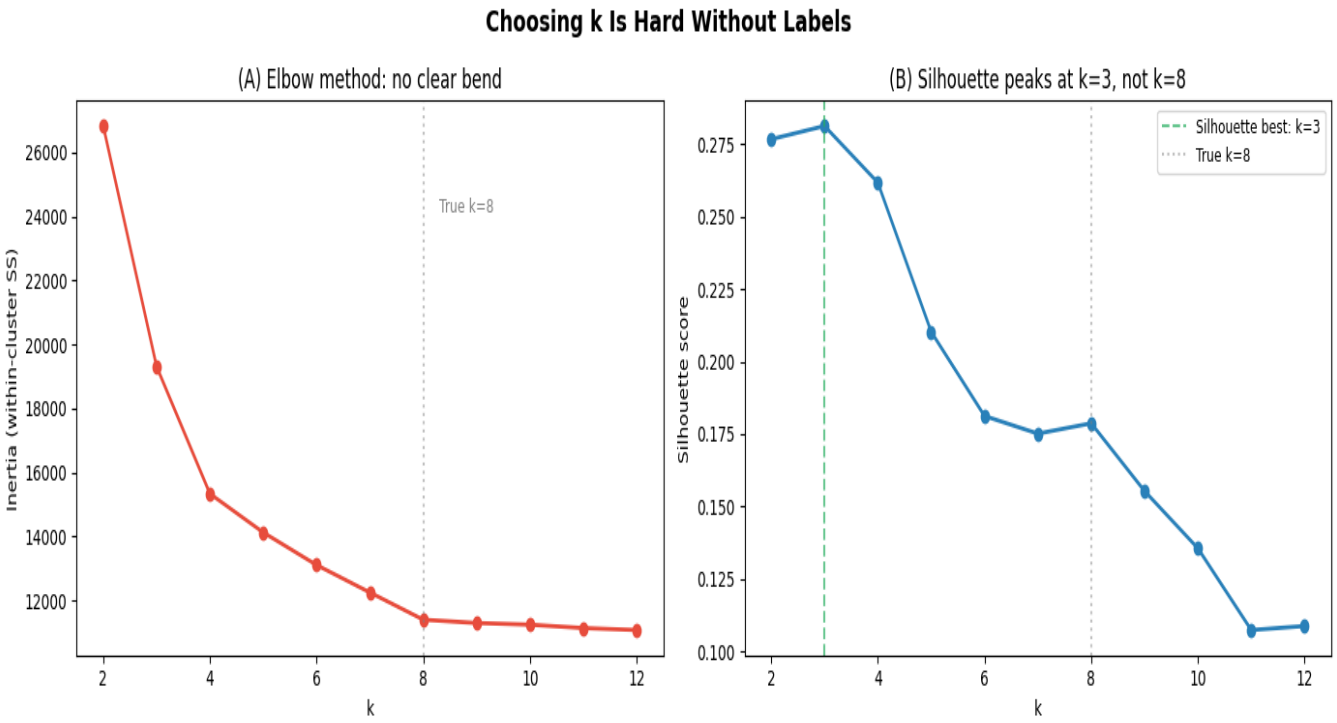

Figure 5. Choosing k without labels. (A) Elbow method: inertia decreases smoothly — no clear bend at k=8. (B) Silhouette score: peaks at k=3, not the true k=8. Grey dashed line marks the correct k. Neither heuristic finds the right answer.

5. What Clustering Cannot Find

Why does k-means completely fail to discover healthy vs impaired?

Every supervised classifier from Weeks 5–11 could distinguish healthy from impaired subjects with at least 70% accuracy. Can unsupervised clustering find this same distinction? We apply k-means with k=2 to all 480 trials and compare the cluster assignments with the healthy/impaired labels. The result: ARI ≈ 0 — complete failure. The two clusters do not correspond to healthy vs impaired at all (Figure 6A).

Why? Impairment in our dataset is a subtle shift in synergy structure — the impaired subjects' muscles are driven by 2 merged synergies instead of 3 (Week 4). Both groups produce similar neural firing rates for the same direction; the impairment signal is small compared to the direction signal. K-means finds the dominant source of variance in the data, which is direction, not impairment.

A subtler trap: cluster only the impaired subjects and look for "impairment subtypes." With k=4, the clusters correlate with reach direction (ARI=0.53) but not with patient identity (ARI≈0). Figure 6B shows this: what appears to be "subtypes of impairment" is actually just the direction structure reappearing. This is a critical warning for clinical researchers using clustering for patient stratification: always check whether your clusters reflect the signal you care about or merely the dominant source of variance.

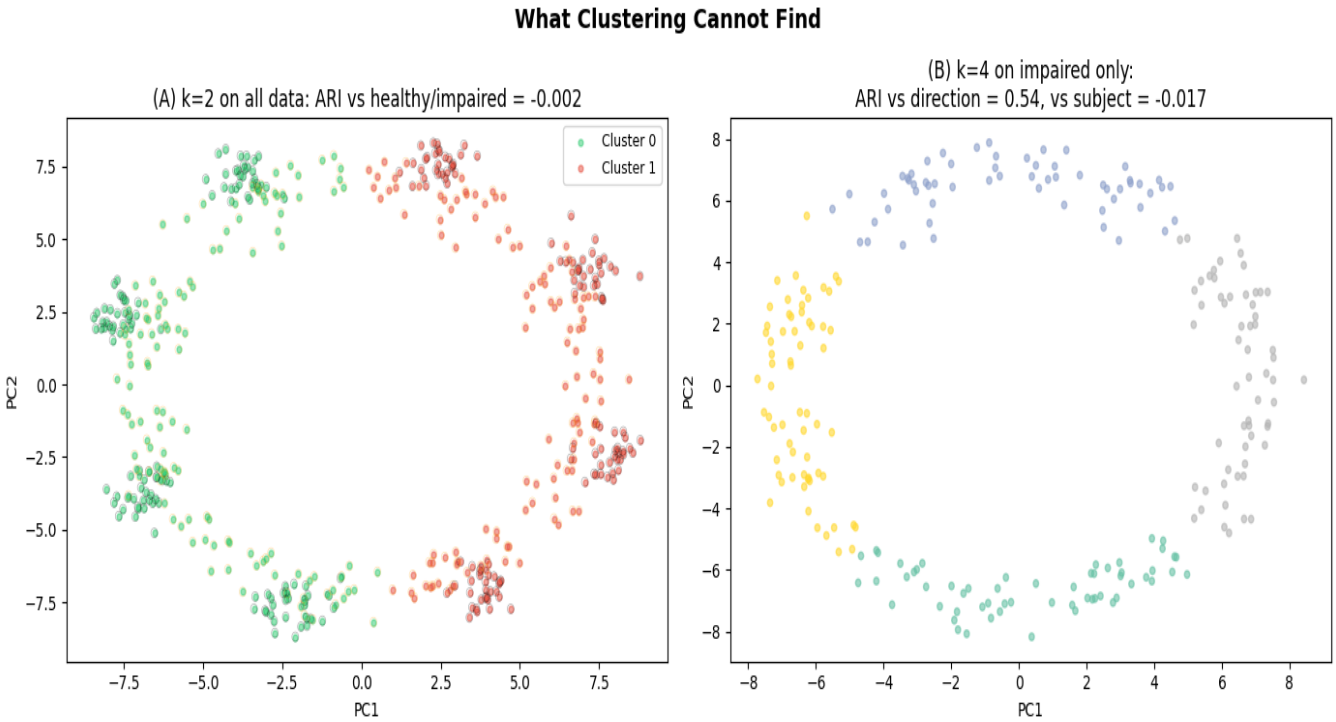

Figure 6. What clustering cannot find. (A) k=2 on all data: clusters do not match healthy/impaired (ARI≈0). (B) k=4 on impaired subjects only: clusters match reach direction (ARI=0.53), not patient subtypes (ARI≈0). Clustering finds the dominant structure, which is not always the structure you want.

In both cases — the k=2 failure on all data and the subtypes illusion on impaired data — the root cause is the same: direction is a much larger signal than impairment. Figure 7 puts numbers on this. Between-direction variance is 357× larger than between-group (healthy/impaired) variance

for neural features, and 54× larger for EMG. K-means partitions the space along the largest source of variance, so it finds direction every time. The impairment signal is buried underneath, invisible to any method that does not know to look for it.

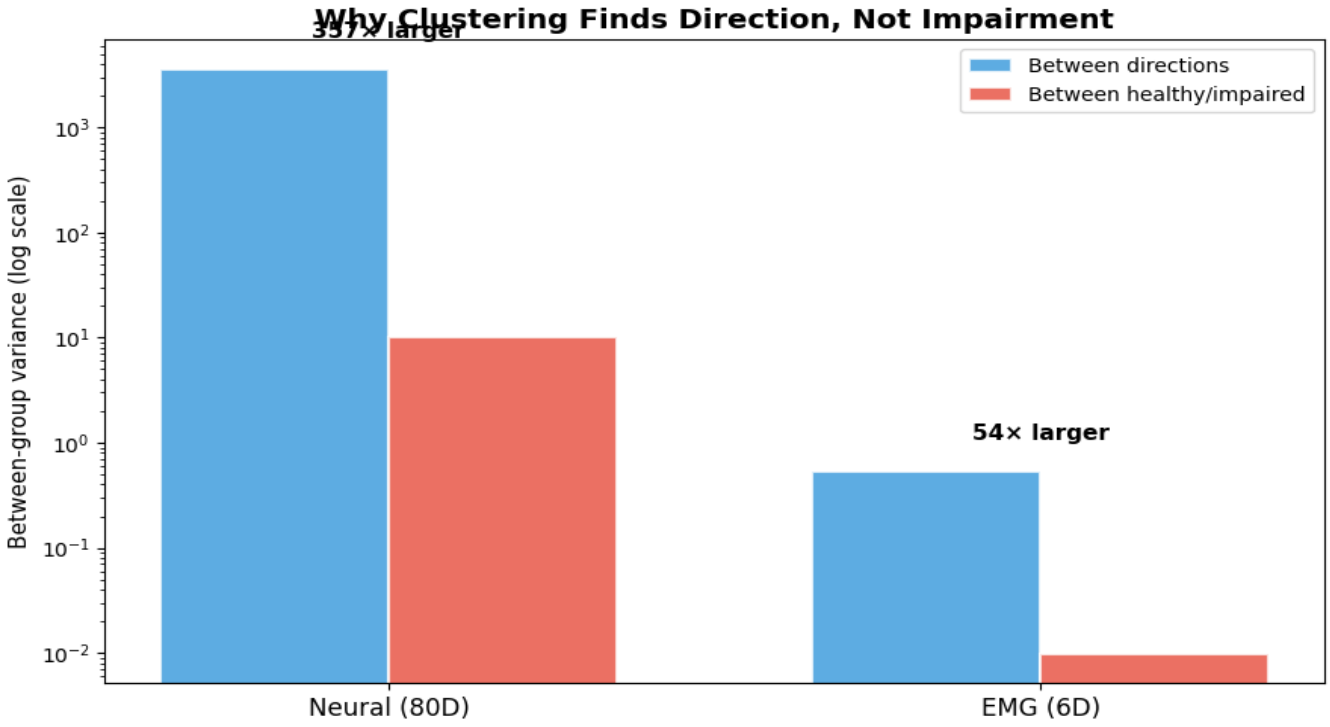

Figure 7. The variance hierarchy explains the failure. Between-direction variance (blue) is many times larger than between-group variance (red) in both neural and EMG features. K-means finds direction because direction dominates the variance.

6. Gaussian Mixture Models — Soft Clustering

What if trials can belong to multiple clusters with different probabilities?

K-means assigns each trial to exactly one cluster: a hard decision. But a reach at 22.5° — halfway between the 0° and 45° targets — might genuinely share characteristics of both directions. A Gaussian Mixture Model (GMM) replaces the hard assignment with probabilities: this trial has a 60% chance of belonging to cluster 1 and a 40% chance of belonging to cluster 2.

GMM assumes each cluster is a Gaussian distribution with its own mean and covariance. This connects directly to two previous weeks. In Week 7, Naive Bayes modelled each class as a Gaussian and used Bayes' theorem to compute posterior probabilities — P(direction | neural activity). GMM does the same thing, but without labels: it discovers the Gaussians from the data using an algorithm called Expectation-Maximisation (EM). In Week 10, LDA assumed all classes share the same covariance (equal-covariance assumption), while QDA allowed each class its own covariance. GMM offers the same choice via covariance_type: 'tied' (one shared covariance, like LDA) vs 'full' (per-cluster covariance, like QDA).

Applying GMM with k=8 to our neural data gives ARI = 0.91 — nearly identical to k-means (0.92). Figure 8 compares the two side by side: k-means produces hard assignments (each trial gets one colour), while GMM produces soft assignments (faded trials have split probabilities). On well-separated data like ours, the results are nearly indistinguishable. Figure 9 shows all four covariance types; all produce ARI = 0.91. Why no improvement? The cosine tuning in our dataset produces roughly spherical, equal-sized clusters. GMM's extra flexibility — different shapes, different sizes — adds nothing when the clusters are already well-behaved. This is the same lesson as QDA vs LDA in Week 10: extra model complexity only helps when the simpler model's assumptions are actually violated.

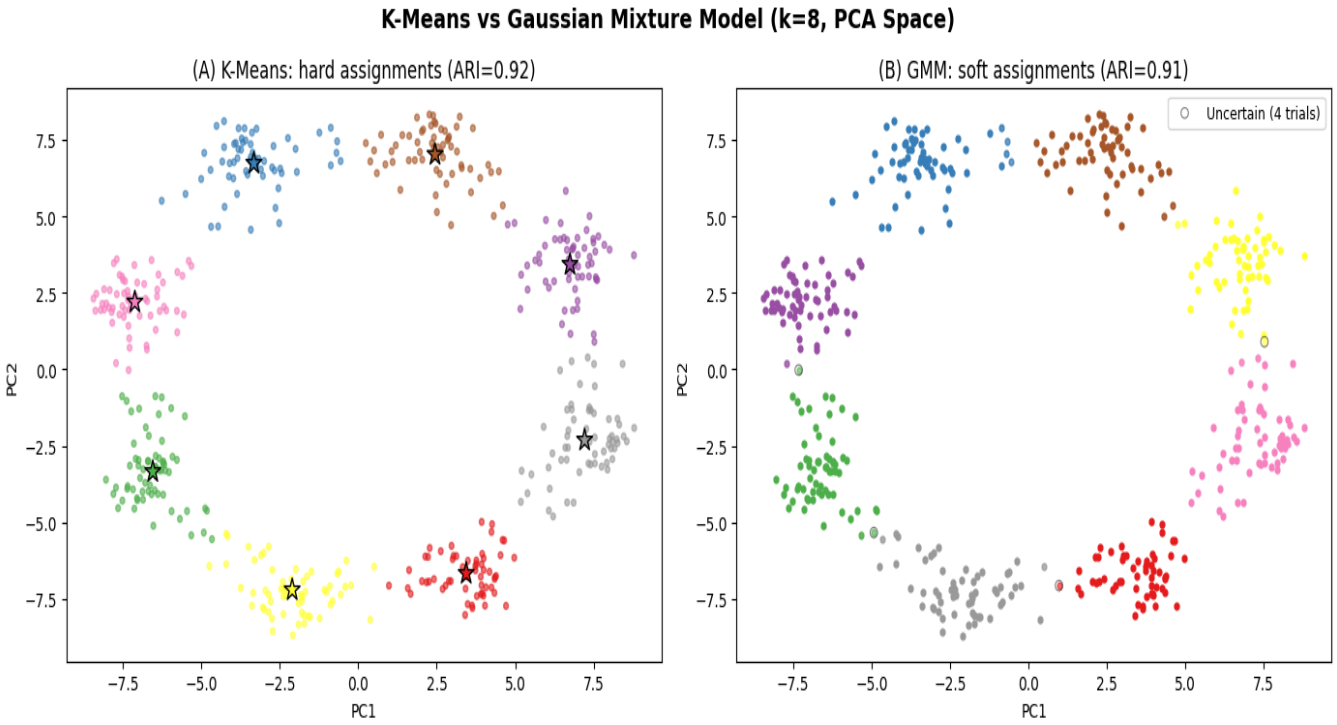

Figure 8. K-means vs GMM (k=8, PCA space). (A) K-means: hard assignments, each trial gets one colour. (B) GMM: soft assignments — faded trials have split probabilities between clusters. Black circles mark uncertain trials (max

probability < 0.7). On well-separated cosine-tuned data, the two methods produce nearly identical results.

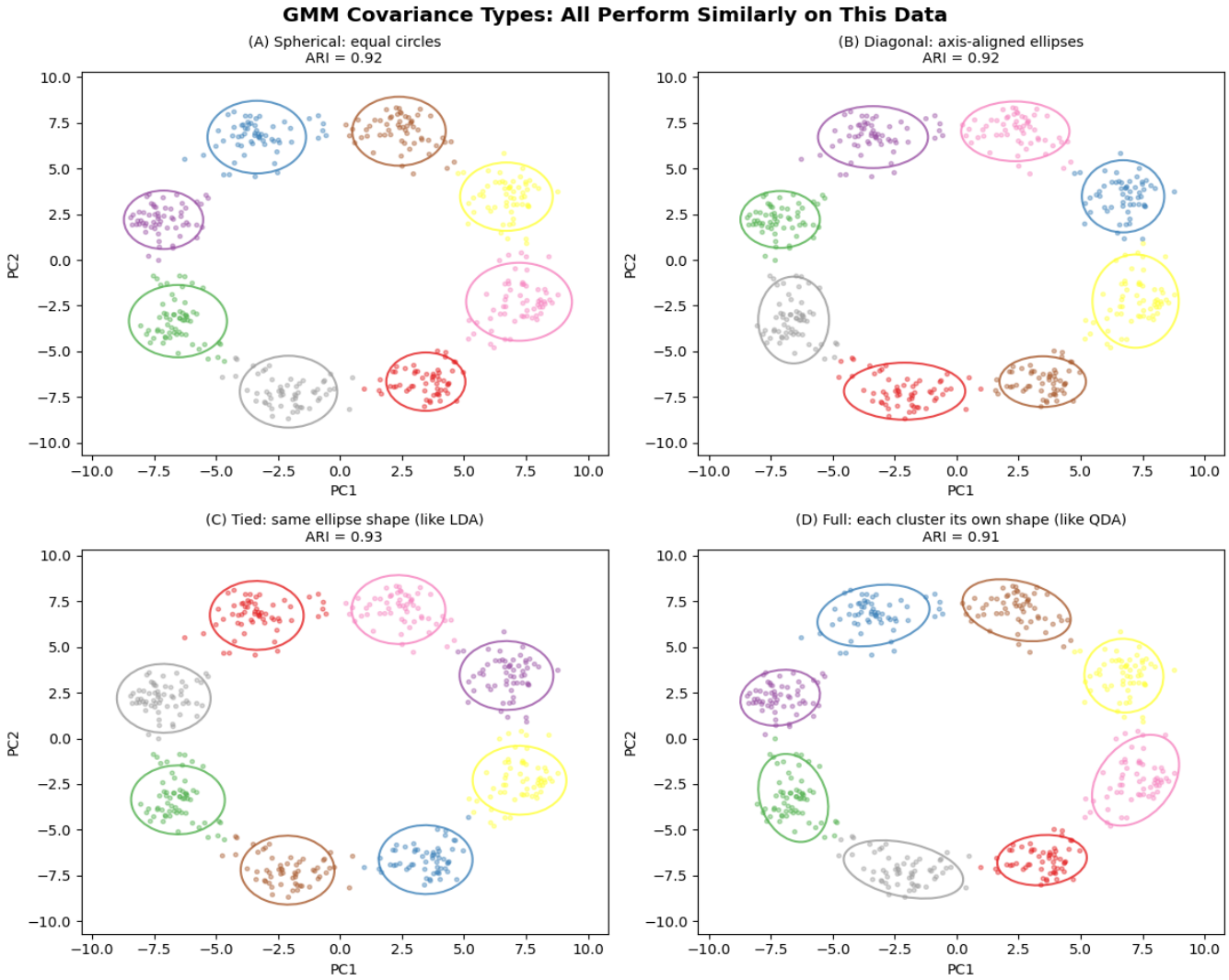

Figure 9. GMM covariance types (k=8, PCA space). (A) Spherical: all clusters are equal circles (like k-means). (B) Diagonal: axis-aligned ellipses. (C) Tied: one shared ellipse shape (like LDA's equal-covariance assumption). (D) Full: each cluster has its own shape (like QDA). Ellipses show 2σ contours. All four types achieve ARI ≈ 0.91 because the clusters are naturally spherical.

When would GMM outperform k-means? Consider a hypothetical scenario. Suppose some reach directions produce tight, consistent neural responses (the subject reaches 0° the same way every time), while others produce highly variable responses (perhaps due to fatigue, or because 90° reaches recruit different motor strategies across trials). The tight cluster would be small and circular; the variable cluster would be elongated and spread along one neural dimension.

Figure 10 simulates this scenario with three clusters of different shapes and sizes. K-means, which carves the space into equal spherical regions, splits the elongated cluster in half (ARI = 0.67). GMM, which fits a separate covariance matrix to each cluster, recognises the elongated shape and recovers all three groups correctly (ARI = 0.97). The covariance ellipses in Panel C show exactly what GMM has learned: a tight circle for the consistent cluster, and tilted ellipses for the variable ones.

Our cosine-tuned reaching data does not show this pattern — all 8 direction clusters are roughly spherical and equal in size, which is why GMM and k-means gave identical results (ARI = 0.92 vs 0.92). But in clinical datasets where different patient subgroups have different amounts of variability, GMM's flexibility can make a real difference. GMM also provides a principled model selection criterion (the Bayesian Information Criterion, or BIC) that can estimate k from the data, though like all model selection methods, it is not infallible.

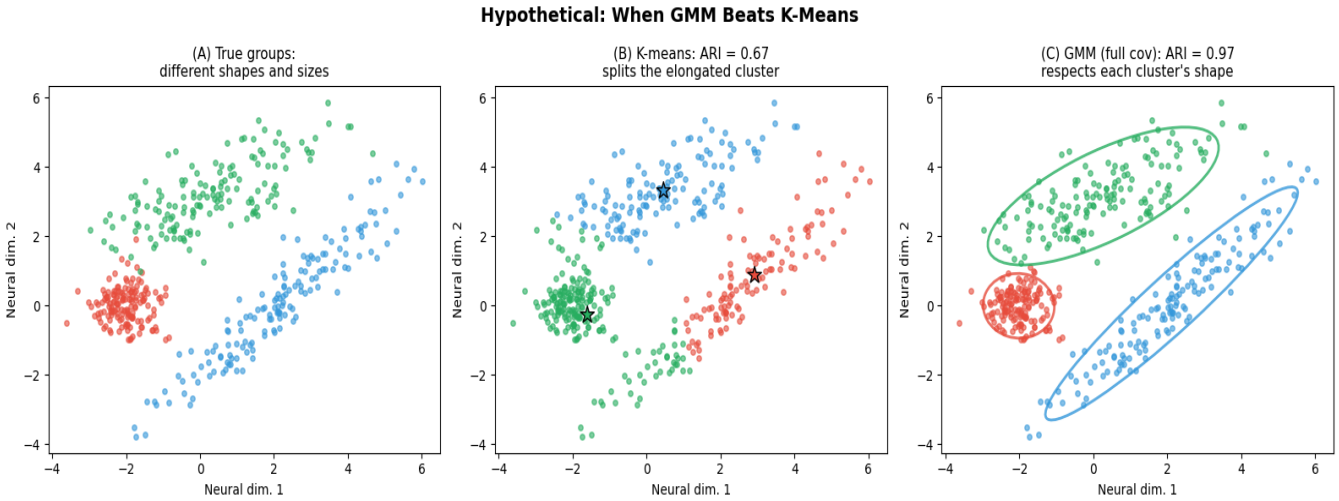

Figure 10. Hypothetical: when GMM beats k-means. (A) Three true groups with different shapes — one tight cluster, one elongated, one tilted. (B) K-means splits the elongated cluster because it assumes all clusters are the same shape (ARI = 0.67). (C) GMM fits a covariance ellipse to each cluster, correctly recovering all three groups (ARI = 0.97).

7. Bringing It Together

Where does unsupervised learning fit in the course?

Weeks 4–12 have now covered both sides of machine learning. PCA (Week 4) and k-means/GMM (this week) are unsupervised: they discover structure without labels. Classification (Weeks 5–11) is supervised: it predicts labels using known ground truth. These are not competing approaches — they answer different questions about the same data.

Figure 11 places all nine methods side by side. Notice that the supervised methods (Weeks 5–11) report LOSO classification accuracy, while the unsupervised methods report ARI. Why can't we use LOSO for clustering? LOSO works by training on 19 subjects and predicting the labels for the held-out 20th subject. That requires labels for the 19 training subjects — without them, there is nothing to train on and no way to measure whether the held-out predictions are correct. Unsupervised methods have no training labels by definition, so LOSO does not apply. Instead, we evaluate clustering by running it on all 480 trials at once and comparing the result with the known labels using ARI. The metrics are therefore not directly comparable — LOSO accuracy measures generalisation to new subjects, while ARI measures agreement with known structure on the same data — but the table highlights the fundamental tradeoff: supervised methods need labels and generalise to new subjects; unsupervised methods need K but not labels, and discover structure across all subjects at once.

Supervised vs Unsupervised: The Complete Picture (Neural 8-Direction)

Method	Type	Neural Dir. Accuracy/ARI	Needs Labels?	Needs K?	Interpretability
NB	Supervised	95.2%	Yes	—	Priors/likelihoods
LR	Supervised	91.2%	Yes	—	Coefficients
KNN	Supervised	94.6%	Yes	—	None
Lin SVM	Supervised	92.7%	Yes	—	Weights
RBF SVM	Supervised	94.6%	Yes	—	None
LDA	Supervised	95.4%	Yes	—	Loadings
RF	Supervised	95.2%	Yes	—	Feature importance
K-means	**Unsupervised**	**ARI=0.92**	**No**	**Yes**	**Cluster centroids**
GMM	**Unsupervised**	**ARI=0.92**	**No**	**Yes**	**Means + covariances**

Figure 11. The complete picture: all nine methods from Weeks 5–12 on the neural 8-direction task. Supervised methods report LOSO accuracy; unsupervised methods (green rows) report ARI. The key columns: supervised methods need labels but not K; unsupervised methods need K but not labels.

When to use clustering:

- When labels do not exist and you want to discover structure. For example, identifying subtypes of motor impairment that have not been previously defined.
- When you want to validate supervised labels. Do the data naturally group into the categories you defined, or is your labelling scheme inconsistent with the actual structure?

• When you want to explore granularity. Are there 4 broad movement categories or 8 fine-grained directions? Clustering at different k reveals different levels of structure.

When NOT to use clustering:

• When you have labels and want to maximise classification accuracy. Supervised methods are always more powerful when ground truth exists, because they focus on the signal you care about rather than the dominant variance.

• When the signal of interest is not the dominant source of variance. Clustering found direction (the largest signal) but missed impairment (the smaller signal). If your clinical question involves a subtle effect, supervised methods are essential.

• When you need to predict labels for individual new trials. K-means can assign new trials to existing clusters, but it was not optimised for prediction accuracy.

What we learned this week

• K-means is fast and intuitive. On our neural data it recovers 8 reach directions with ARI=0.92 — without ever seeing the labels.

• Choosing k is hard. Neither the elbow method nor the silhouette score correctly identified k=8. The right k depends on the question, not just the data.

• Clustering finds the dominant structure, which may not be the structure you care about. Direction dominated variance; impairment was invisible to k-means.

• GMM extends k-means with soft assignments and per-cluster covariance. It connects to NB (Week 7, supervised Gaussian model) and LDA/QDA (Week 10, equal vs unequal covariance). On our data, GMM performs identically to k-means because the clusters are already spherical.

• PCA (Week 4) finds axes; clustering finds groups; classification (Weeks 5–11) predicts labels. Each answers a different question — the right tool depends on what you are trying to learn from the data.

References

Lloyd SP. Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137, 1982.

Arthur D, Vassilvitskii S. K-means++: The advantages of careful seeding. Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 1027–1035, 2007.

Dempster AP, Laird NM, Rubin DB. Maximum likelihood from incomplete data via the EM algorithm. Journal of the Royal Statistical Society: Series B, 39(1), 1–38, 1977.

Tresch MC, Cheung VCK, d’Avella A. Matrix factorization algorithms for the identification of muscle synergies: evaluation on simulated and experimental data sets. Journal of Neurophysiology, 95(4), 2199–2212, 2006.

Georgopoulos AP, Kalaska JF, Caminiti R, Massey JT. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. Journal of Neuroscience, 2(11), 1527–1537, 1982.